National Sun Yat-sen University

Department of Electrical Engineering

114-2 Practical Digital System Design

HW2 Voting and Median Circuit Design

Name: Enping, Chung

Student ID: B133015012

Table of Contents

Part I.Voting

I.Design Process

Goal：A voting circuit of this type takes five 5-bit one-hot inputs, where each input has exactly one bit set to logic 1, representing a selected position. The circuit first uses combinational logic, such as adders, to count how many times each bit position is selected across the five inputs, effectively generating five vote totals. These totals are then compared using comparators to determine which position has the highest count. In cases where two or more positions have equal counts, an arbitration or priority mechanism is applied to select one based on a predefined rule, typically favoring the higher-order bit. Once the winning position is identified, encoding and decoding logic (or multiplexers) are used to produce a 5-bit one-hot output corresponding to that position, while the associated vote total is converted into a 3-bit binary value representing how many times that input occurred.

Design I

Design I works by first counting how many times each bit position is selected across the five one-hot inputs, then choosing the position with the highest count and outputting both that position and its frequency. Specifically, it begins by computing five separate vote counts, one for each bit position (from bit 0 to bit 4), where each count represents how many inputs have a logic 1 in that position. This counting process can be viewed as a set of parallel addition functions, where each function sums the corresponding bits from all inputs, such as:

c0 = a0[0] + a1[0] + a2[0] + a3[0] + a4[0]

c1 = a0[1] + a1[1] + a2[1] + a3[1] + a4[1]

c2 = a0[2] + a1[2] + a2[2] + a3[2] + a4[2]

c3 = a0[3] + a1[3] + a2[3] + a3[3] + a4[3]

c4 = a0[4] + a1[4] + a2[4] + a3[4] + a4[4]

After obtaining these counts, the algorithm performs a comparison function to determine the maximum value among them. This can be interpreted as a max-selection function:

max_count = max(c0, c1, c2, c3, c4)

If multiple positions have the same highest count, a priority selection function is applied that favors the higher-indexed bit. This priority logic ensures deterministic behavior and can be expressed as a sequence of conditional comparisons starting from the highest bit:

if (c4 >= c3, c2, c1, c0) select bit 4

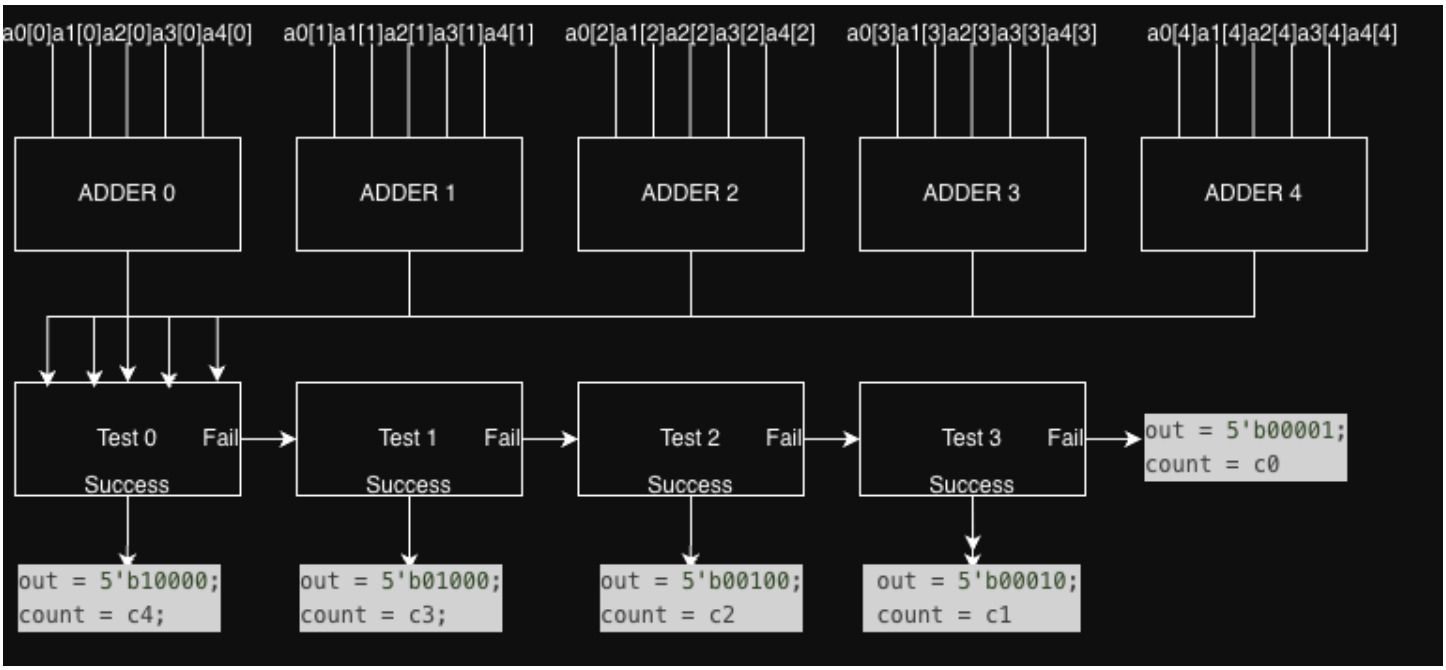
else if (c3 >= c2, c1, c0) select bit 3

else if (c2 >= c1, c0) select bit 2

else if (c1 >= c0) select bit 1

else select bit 0

Finally, an encoding function converts the selected bit position into a one-hot 5-bit output, while a routing function assigns the corresponding vote count to the output



Design II

Deign II implements a combinational 5-input voting circuit that takes five 5-bit one-hot inputs and determines which bit position (0–4) is selected most frequently. Each input is first converted from a one-hot vector into a 3-bit index representing the active bit position. The design then computes five vote counts, where each count represents how many of the inputs correspond to a given index from 0 to 4. After obtaining these counts, a tournament-style comparison tree is used to find the maximum value efficiently by comparing pairs of counts step by step until a final winner is selected. If two counts are equal, a tie-breaking rule is applied that selects the higher index to ensure deterministic behavior. The max2_pick module performs each pairwise comparison by choosing the larger count and, in the case of a tie, the larger index. Finally, the winning index is converted back into a one-hot 5-bit output using a left shift operation, and the corresponding maximum count is assigned to the count output, producing both the most popular bit position and its frequency.

II.Design Comparison

Design II is better than Design I because it replaces the brute-force bit-column adders plus a long priority if/else chain with a more structured datapath: it first encodes each one-hot input into a 3-bit index, then computes bucket tallies using parallel equality checks, and finally selects the winner with a small 4-stage max tournament (max2_pick). This makes the design more regular, easier to synthesize for speed, and avoids the wide multi-way comparisons of the original code, while preserving the same tie rule and I/O behavior.

III.Testbench

The testbench first defines the inputs, outputs, and extra signals used for verification. The outputs from the UUT are out (the winning one-hot bit position) and count (how many inputs voted for that position). It also defines expected values (exp_out, exp_count) and counters to track total tests and errors. The main verification logic is in calculate_expected_value, which simulates the voting process in software. It counts how many inputs select each bit position (0 to 4), then chooses the highest count. If there is a tie, the higher bit position wins because of the priority order in the if-else chain. It then sets the expected output and count.The run_all_cases run first, using a For loop, all legal inputs are tested with no delay. Then, run_case task applies inputs to the UUT, computes the expected result, waits for the circuit to settle, and compares the UUT output with the expected values. It prints PASS if they match or FAIL if they don't, while also tracking errors. Finally, the initial block runs multiple test cases, including normal, edge, and corner cases, and prints a summary of results at the end. Overall, the testbench verifies that the voting circuit behaves correctly under all conditions, including tie-breaking scenarios.

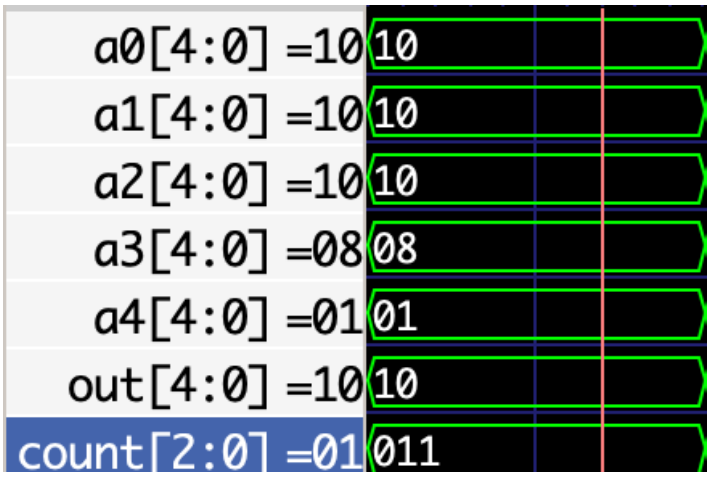
Terminal output:

```
[ALL CASES]------------------------------------------------------------------------------------------
ALL SUMMARY: PASS (3125 tests)
[SELECTED CASES]-------------------------------------------------------------------------------------
 time  |  a0    a1    a2    a3    a4   | exp_out  out   | exp_count  got_count | Result | Description
-------------------------------------------------------------------------------------------------------
[MAIN FUNCTION CASES]--------------------------------------------------------------------------------
    10 | 10000 10000 10000 01000 00001 | 10000    10000 | 3(011)   3(011) | PASS   | main_1_example_from_TA
    20 | 00010 00010 00010 00010 00100 | 00010    00010 | 4(100)   4(100) | PASS   | main_2_clear_majority
    30 | 10000 01000 00100 00001 00001 | 00001    00001 | 2(010)   2(010) | PASS   | main_3_three_way_bottom
[EDGE CASES]-----------------------------------------------------------------------------------------
    40 | 10000 01000 00100 00010 00001 | 10000    10000 | 1(001)   1(001) | PASS   | edge_1_all_candidates_tie
    50 | 10000 10000 01000 01000 00001 | 10000    10000 | 2(010)   2(010) | PASS   | edge_2_two_way_top_tie
    60 | 00001 00001 00001 00001 00001 | 00001    00001 | 5(101)   5(101) | PASS   | edge_3__all_same
[CORNER CASES]---------------------------------------------------------------------------------------
    70 | 00001 00001 00001 00001 00001 | 00001    00001 | 5(101)   5(101) | PASS   | corner_1_lowest_unanimous
    80 | 00010 00010 00001 00001 00100 | 00010    00010 | 2(010)   2(010) | PASS   | corner_2_low_bit_tie
    90 | 10000 01000 00100 00100 00010 | 00100    00100 | 2(010)   2(010) | PASS   | corner_3_valid_spread_repea
-------------------------------------------------------------------------------------------------------
SUMMARY: PASS (9/9)
```

IV.Waveform Analysis

(10=10000, 08=01000, 04=00100, 02=00010, 01=00001)

Main Function Cases:



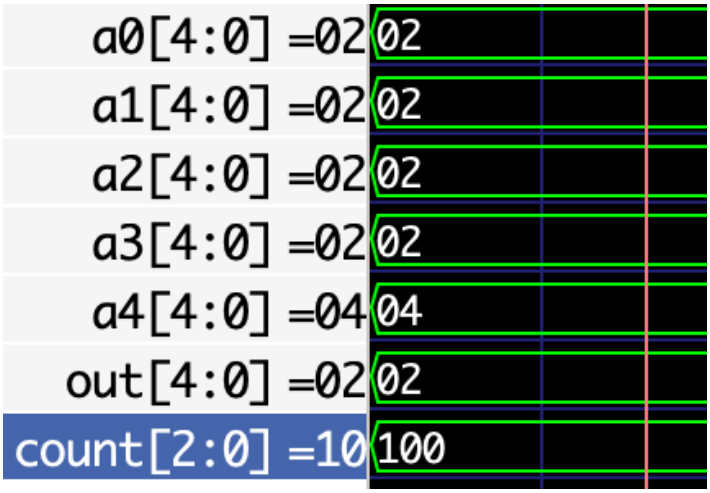
a0 = 10000 a1 = 10000 a2 = 10000

a3 = 01000 a4 = 00001

---

out = 10000 count = 3

main_1_example_from_TA



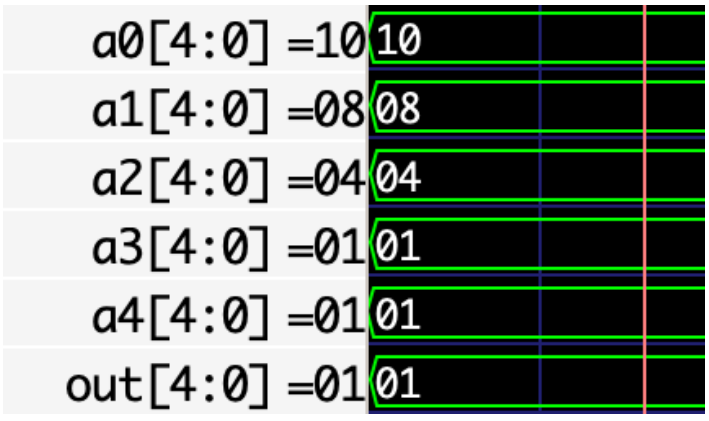
a0 = 00010 a1 = 00010 a2 = 00010

a3 = 00010 a4 = 00100

---

out = 00010 count = 4

main_2_clear_majority



a0 = 10000 a1 = 01000 a2 = 00100

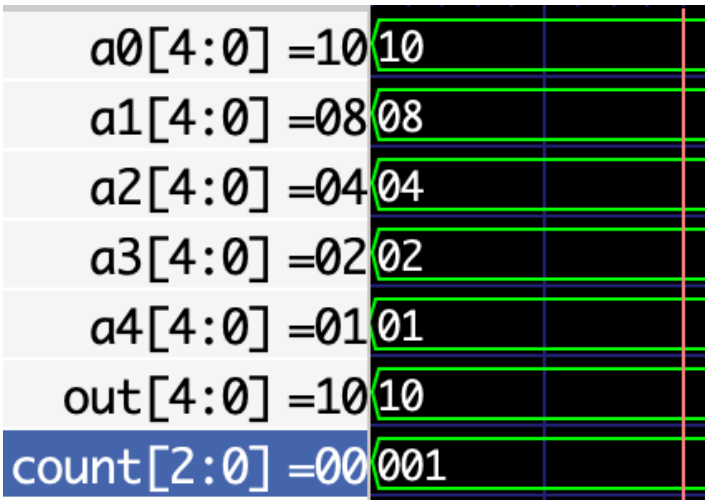
a3 = 00001 a4 = 00001

---

out = 00001 count = 2

main_3_three_way_bottom

Edge Cases:



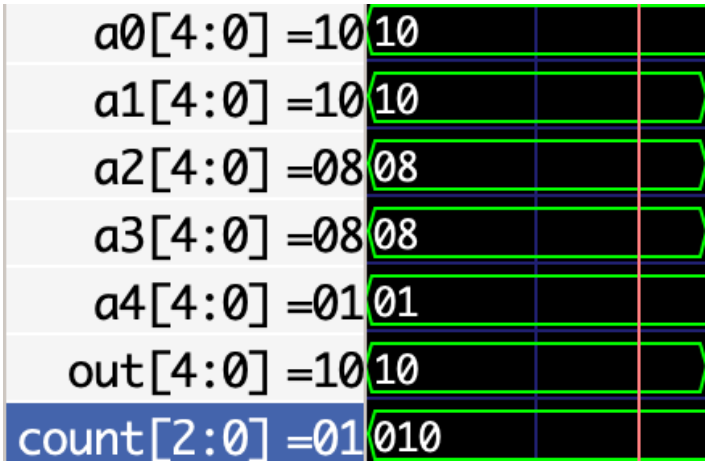
a0 = 10000 a1 = 01000 a2 = 00100

a3 = 00010 a4 = 00001

---

out = 10000 count = 1

edge_1_all_candidates_tie



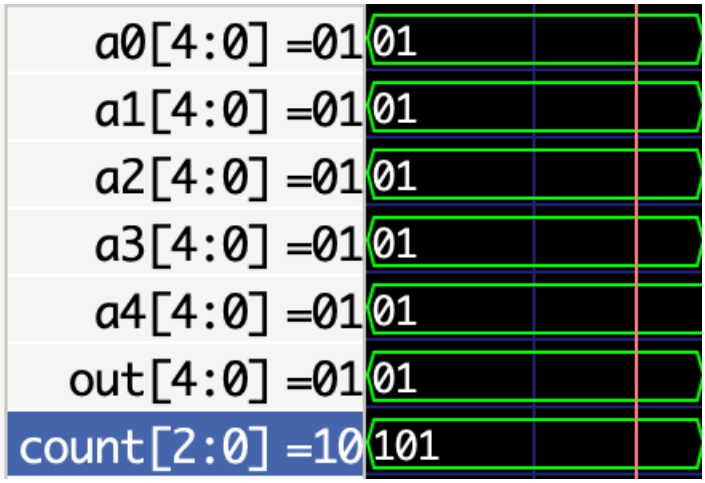
a0 = 10000 a1 = 10000 a2 = 01000

a3 = 01000 a4 = 00001

---

out = 10000 count = 2

edge_2_two_way_top_tie



a0 = 00001 a1 = 00001 a2 = 00001

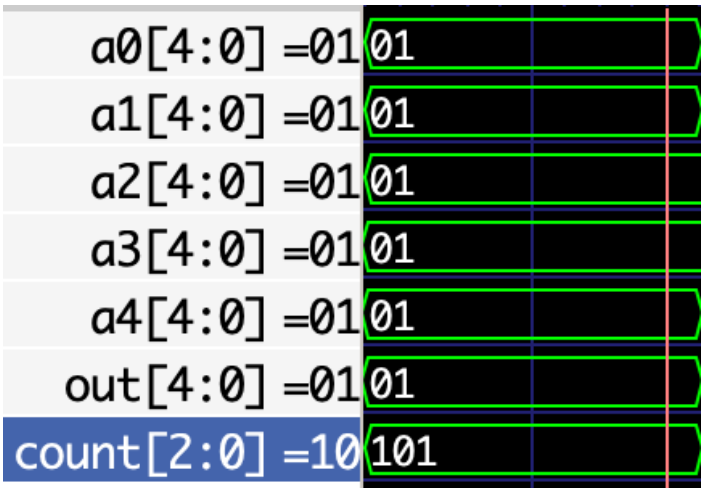
a3 = 00001 a4 = 00001

---

out = 00001 count = 5

edge_3_all_same

Corner Cases:



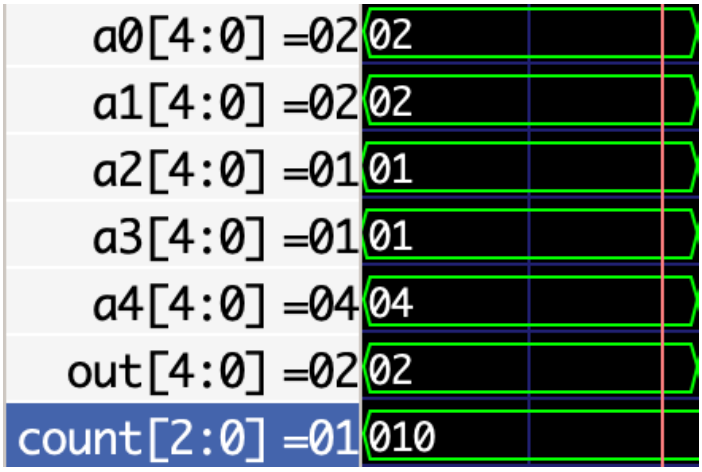
a0 = 00001 a1 = 00001 a2 = 00001

a3 = 00001 a4 = 00001

---

out = 00001 count = 5

corner_1_lowest_unanimous



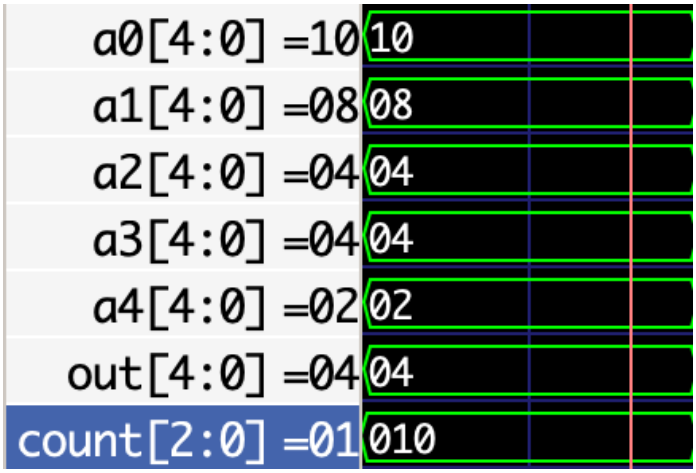
a0 = 00010 a1 = 00010 a2 = 00001

a3 = 00001 a4 = 00100

---

out = 00010 count = 2

corner_2_low_bit_tie



a0 = 10000 a1 = 01000 a2 = 00100

a3 = 00100 a4 = 00010

---

out = 00100 count = 2

corner_3_valid_spread_repeat

V.Circuit Analysis

For circuit analysis, since both designs were written in behavior level, we used open source program Yosys to synthesize the circuit and extimate the amount of logic gates used.

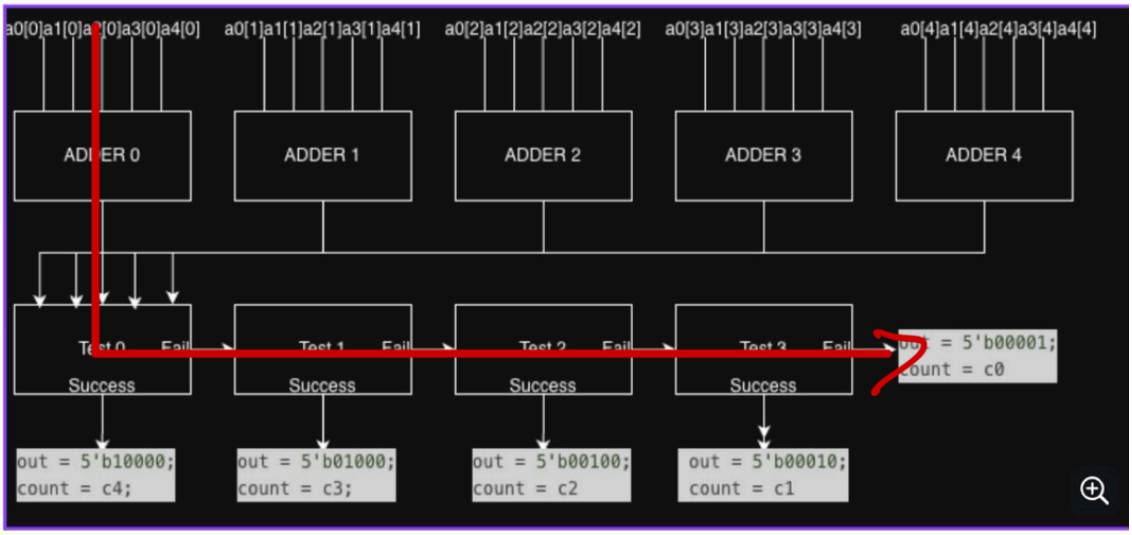
For Design I

Terminal Output:

```
=== voting ===

        +----------Local Count, excluding submodules.
        |
      607 wires
      633 wire bits
      230 public wires
      256 public wire bits
        7 ports
       33 port bits
      377 cells
      113   $and
       12   $mux
      154   $not
       38   $or
       60   $xor
```

Longest Critical Path:



For Design II

Terminal Output:

```
=== design hierarchy ===

        +----------Count including submodules.
        |
      316 voting
       40 max2_pick

        +----------Count including submodules.
        |
      316 wires
      416 wire bits
       44 public wires
      144 public wire bits
       31 ports
      105 port bits
        - memories
        - memory bits
        - processes
      316 cells
       80   $_ANDNOT_
       48   $_MUX_
       22   $_NAND_
       15   $_NOR_
        2   $_NOT_
       42   $_ORNOT_
       38   $_OR_
       21   $_XNOR_
       48   $_XOR_
        4 submodules
        4   max2_pick
```

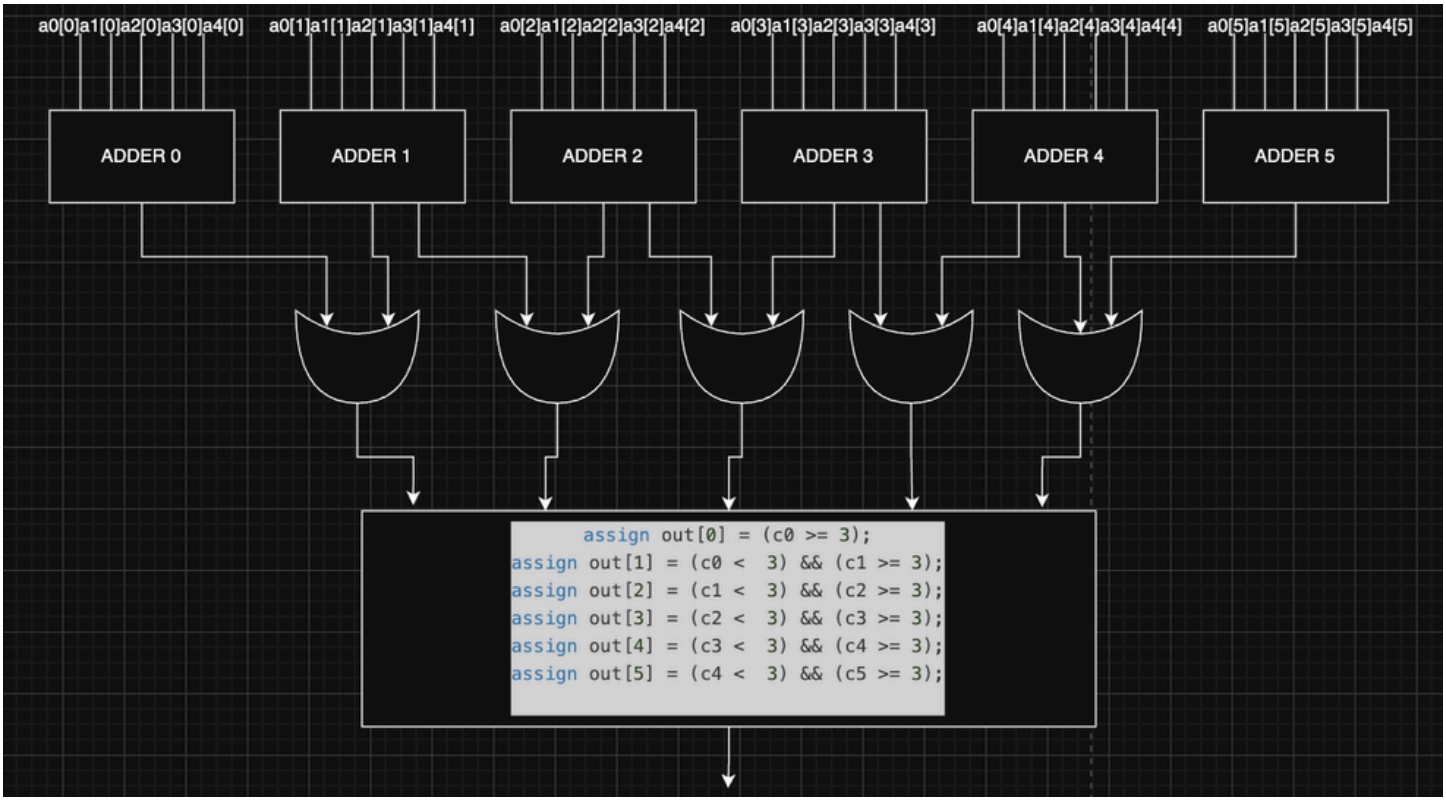
PART II. Median

I.Design Process

Goal：The median module takes five 6-bit inputs and analyzes them bit-by-bit across all five values. For each bit position (0 to 5), it first counts how many of the five inputs have a logic ‘1’ at that position (s0 to s5). It then forms a cumulative sum across bit positions (c0 to c5), effectively building a running total of how many ones have been seen up to each bit index. Finally, it generates a one-hot 6-bit output where exactly one bit is set: the output indicates the first bit position where the cumulative count reaches or exceeds 3 (a majority of the five inputs). In other words, the module finds the “median” bit position in terms of where the majority of ones are concentrated across the inputs.

Design

The module is designed to take five 6-bit inputs and produce a 6-bit one-hot output that indicates the “median position” based on where the accumulation of 1s crosses a midpoint threshold.
First, the design treats each bit position independently. For each of the six bit positions (from bit 0 to bit 5), it counts how many of the five inputs have a logic 1 in that position. This produces six small count values, each representing how many inputs are “active” in that bit position.
Next, instead of stopping at these individual counts, the design builds a running cumulative total across bit positions. Starting from bit 0, it progressively adds the counts from lower to higher bit positions. This creates a prefix-style accumulation that represents how many total 1s have appeared up to each bit position.
Finally, the output is generated by finding the first bit position where this cumulative total reaches or exceeds the value 3. Since there are five inputs, 3 represents the majority (or median threshold). The output is one-hot, meaning only one bit is set to 1, corresponding to the first position where the accumulated count crosses this threshold.



II.Testbench

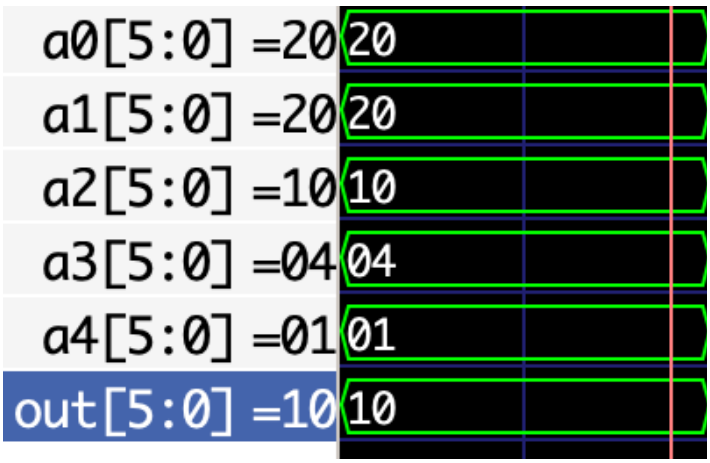
The testbench first defines the inputs, outputs, and signals used for verification. The inputs are five 6-bit one-hot values (a0 to a4), and the output out is the median result in one-hot form. It also defines expected values (exp_out, exp_med) and counters for total tests and errors.
The main verification logic is in calculate_expected_value, which acts as a golden model. It counts how many inputs select each value (0–5), forming a histogram, then computes cumulative sums. The median is chosen as the smallest value where the cumulative count reaches at least three, since there are five inputs. This value is then converted back to one-hot form for comparison.
The run_all_cases task performs exhaustive testing using nested loops to try all possible input combinations, ensuring full coverage of the design.
The run_case task applies specific inputs, computes the expected result, waits for the circuit to settle, and compares it with the DUT output. It prints PASS or FAIL and updates the error count.
Finally, the initial block runs directed, edge, and corner cases, then prints a summary of results. Overall, the testbench verifies correct median behavior for all input conditions.

Terminal output:

```
[ALL FUNCTION CASES]----------------------------------------------------------------------------------------------------
ALL SUMMARY: PASS (7776 tests)
[SELECTED CASES]--------------------------------------------------------------------------------------------------------
 time  |   a0      a1      a2      a3      a4    | exp_out  out   | exp_med    got_med   | Result | Description
----------------------------------------------------------------------------------------------------------------------
[MAIN FUNCTION CASES]---------------------------------------------------------------------------------------------------
    10 | 100000 100000 010000 000100 000001 | 010000    010000 | 4(100)   4(100) | PASS   | main_1_example_from_TA
    20 | 000001 000010 000100 001000 010000 | 000100    000100 | 2(010)   2(010) | PASS   | main_2_sorted_unique
    30 | 000100 000100 000100 100000 000001 | 000100    000100 | 2(010)   2(010) | PASS   | main_3_clear_middle_repeat
[EDGE CASES]------------------------------------------------------------------------------------------------------------
    40 | 000001 000001 000001 000001 000001 | 000001    000001 | 0(000)   0(000) | PASS   | edge_1_all_same_lowest
    50 | 100000 100000 100000 100000 100000 | 100000    100000 | 5(101)   5(101) | PASS   | edge_2_all_same_highest
    60 | 000001 000010 000100 001000 100000 | 000100    000100 | 2(010)   2(010) | PASS   | edge_3_wide_spread
[CORNER CASES]----------------------------------------------------------------------------------------------------------
    70 | 010000 010000 000001 100000 000010 | 010000    010000 | 4(100)   4(100) | PASS   | corner_1_two_equal_median
    80 | 001000 000100 001000 010000 000001 | 001000    001000 | 3(011)   3(011) | PASS   | corner_2_middle_dominates
    90 | 100000 010000 010000 001000 000001 | 010000    010000 | 4(100)   4(100) | PASS   | corner_3_top_heavy
----------------------------------------------------------------------------------------------------------------------
SUMMARY: PASS (9/9)
```

III.Waveform Analysis

(20= 100000, 10=010000, 08=001000, 04=000100, 02=000010, 01=000001)

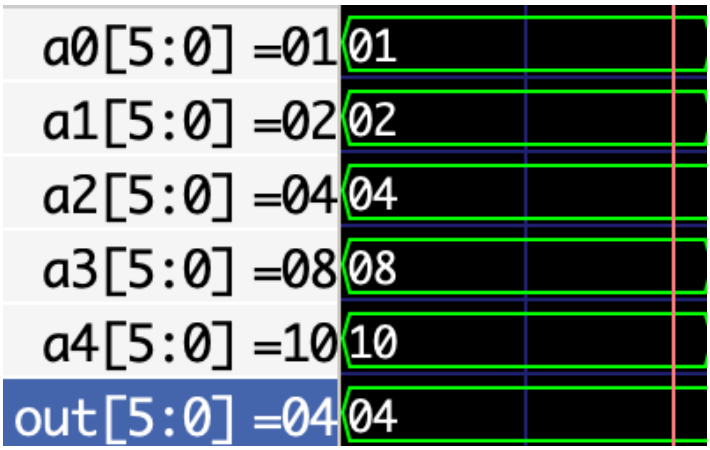


a0 = 100000 a1 = 100000 a2 = 010000

a3 = 000100 a4 = 000001

out = 010000

main_1_example_from_TA

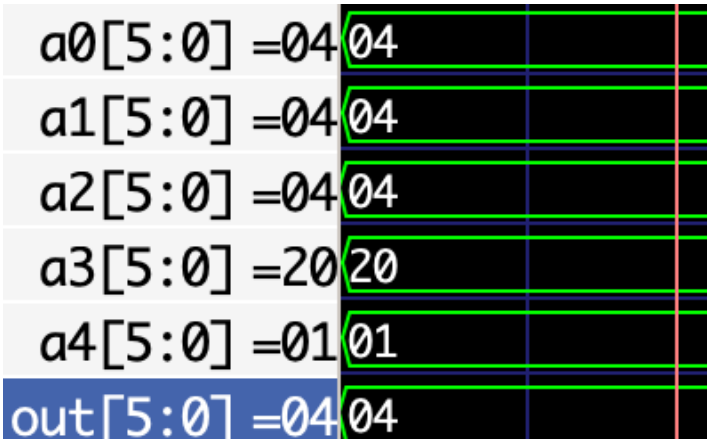


a0 = 000001 a1 = 000010 a2 = 000100

a3 = 001000 a4 = 010000

out = 000100

main_2_sorted_unique

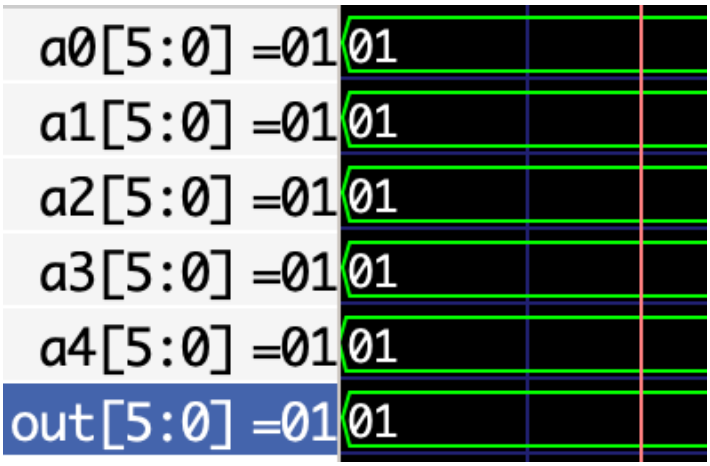


a0 = 000100 a1 = 000100 a2 = 000100

a3 = 100000 a4 = 000001

out = 000100

main_3_clear_middle_repeat

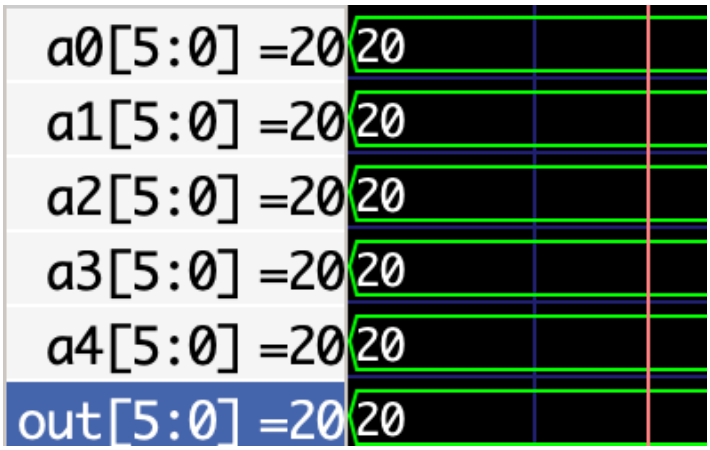


a0 = 000001 a1 = 000001 a2 = 000001

a3 = 000001 a4 = 000001

out = 000001

edge_1_all_same_lowest

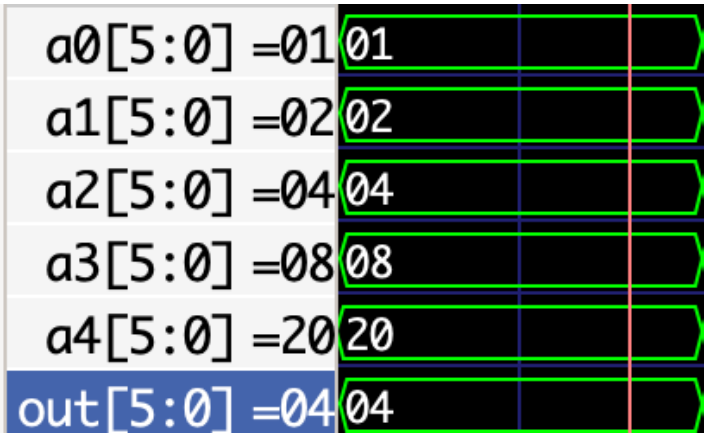


a0 = 100000 a1 = 100000 a2 = 100000

a3 = 100000 a4 = 100000

out = 100000

edge_2_all_same_highest

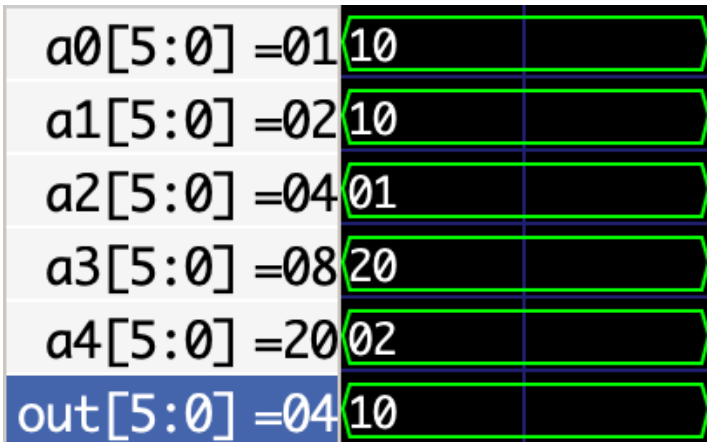


a0 = 000001 a1 = 000010 a2 = 000100

a3 = 001000 a4 = 100000

out = 000100

edge_3_wide_spread

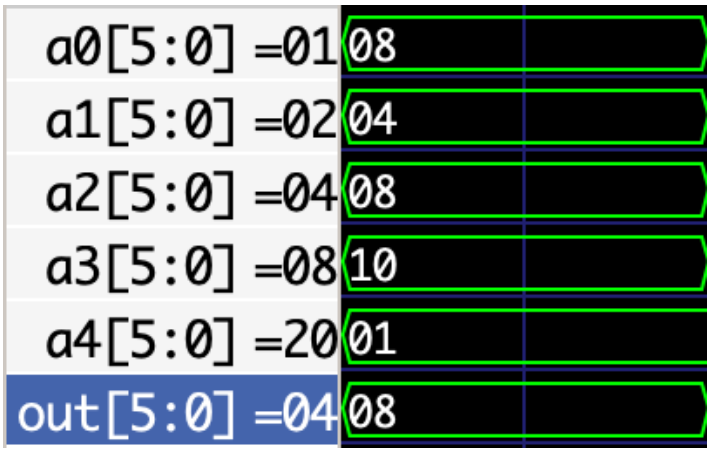


a0 = 010000 a1 = 010000 a2 = 000001

a3 = 100000 a4 = 000010

out = 010000

corner_1_two_equal_median

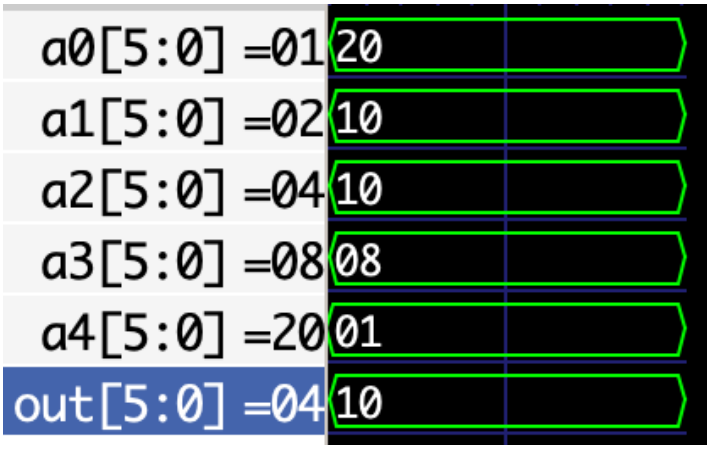


a0 = 001000 a1 = 000100 a2 = 001000

a3 = 010000 a4 = 000001

out = 001000

corner_2_middle_dominates



a0 = 100000 a1 = 010000 a2 = 010000

a3 = 001000 a4 = 000001

out = 010000

corner_3_top_heavy

IV.Circuit Analysis

For circuit analysis, since both designs were written in behavior level, we used open source program Yosys to synthesize the circuit and extimate the amount of logic gates used.

Terminal Output:

```
=== median ===

        +----------Local Count, excluding submodules.
        |
      136 wires
      166 wire bits
        6 public wires
       36 public wire bits
        6 ports
       36 port bits
      136 cells
       49   $_ANDNOT_
        9   $_AND_
        6   $_NAND_
        3   $_NOR_
        1   $_NOT_
        7   $_ORNOT_
        6   $_OR_
       13   $_XNOR_
       42   $_XOR_
```

III.Experience and Reflection

Designing the voting and median circuits helped me understand how to turn problem rules into clear combinational logic. In voting, I learned to count one-hot bits across inputs, compare candidate counts, and enforce tie-breaking priority in a deterministic way. In median, I used histogram and prefix-sum logic to find the first value where cumulative count reaches the middle position, which is a clean hardware-friendly way to compute the median of five inputs. I also used Cursor AI to generate a alternative version of each module to compare as a secondary design.

Building thorough testbenches, including directed and all-case testing, was just as important as the RTL itself because it exposed corner cases and confirmed correctness. Overall, this work improved my ability to structure logic step by step, verify behavior systematically, and think about both function and robustness in digital design.

During the circuit analysis part, I realized I couldn’t count the amout of logic gates since I was using behavior level coding, so I had to learn a new open source program Yosys. I am very proud that I have the ability to find open source tools and learn how to use them on my own.